



南京大學

NANJING UNIVERSITY

# 网络层：控制平面

孟晴开

智能软件与工程学院

苏州校区南雍楼西区228

qkmeng@nju.edu.cn , <https://mengqingkai.github.io/>



# Outline

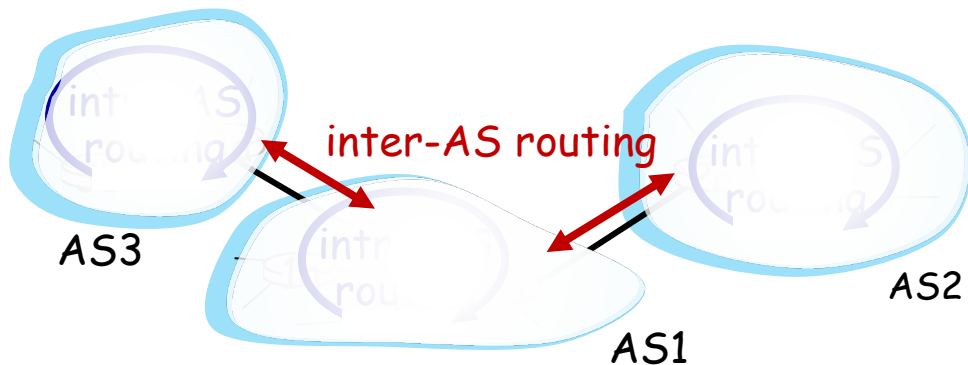
---

- Routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol
- Network management, configuration





# Interconnected ASes



- ✓ **intra-AS** (aka "intra-domain"): routing among routers within same AS ("network")
- ➡ **inter-AS** (aka "inter-domain"): routing among AS'es



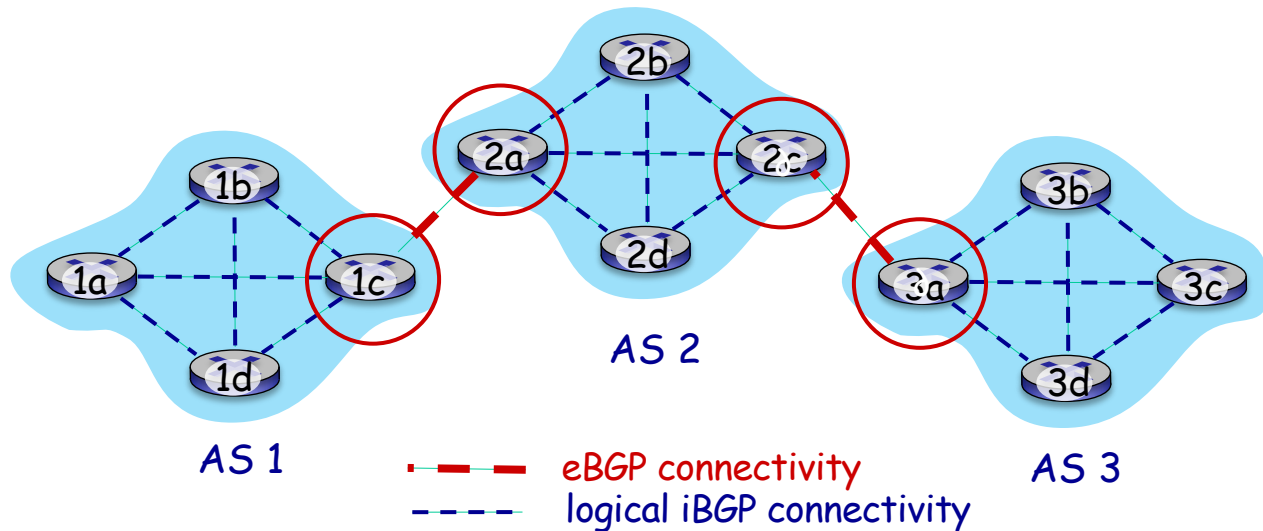
# Internet inter-AS routing: BGP

- **BGP (Border Gateway Protocol):** the de facto inter-domain routing protocol
  - “glue that holds the Internet together”
- allows subnet to advertise its existence, and the destinations it can reach, to rest of Internet: “I am here, here is who I can reach, and how”
- BGP provides each AS a means to:
  - obtain destination network reachability info from neighboring ASes (**eBGP**)
  - **determine routes to other networks** based on reachability information and policy
  - propagate reachability information to all AS-internal routers (**iBGP**)
  - **advertise** (to neighboring networks) destination reachability info





# eBGP, iBGP connections



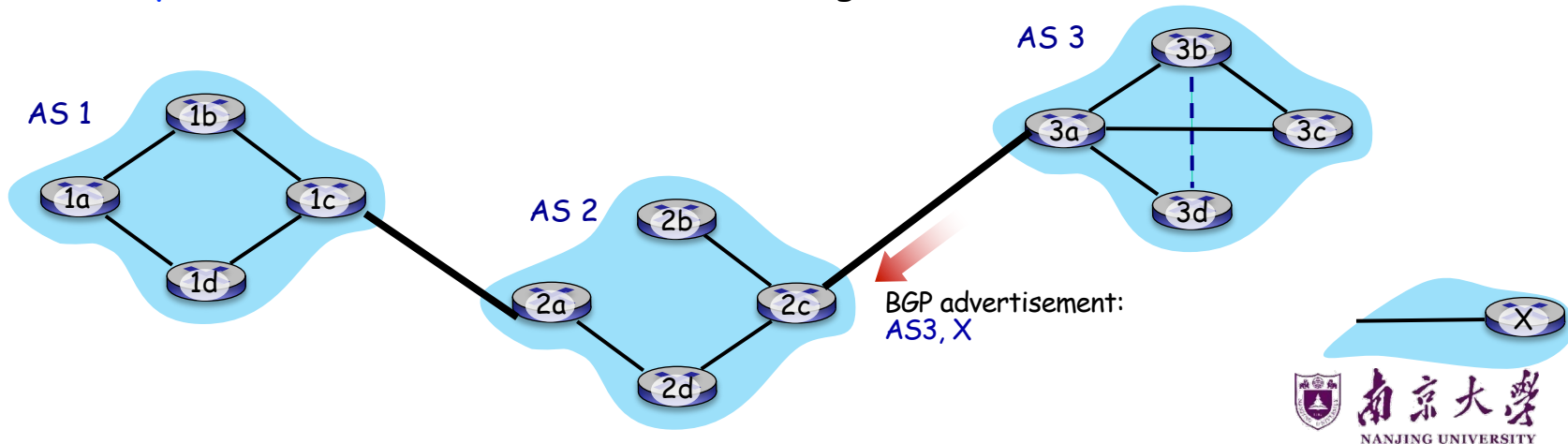
gateway routers run both eBGP and iBGP protocols





# BGP basics

- **BGP session:** two BGP routers ("peers") exchange BGP messages over semi-permanent TCP connection:
  - advertising **paths** to different destination network prefixes (BGP is a "path vector" protocol)
- when AS3 gateway 3a advertises **path AS3,X** to AS2 gateway 2c:
  - AS3 **promises** to AS2 it will forward datagrams towards X





# BGP protocol messages

---

- BGP messages exchanged between peers over TCP connection
- BGP messages [RFC 4371]:
  - **OPEN**: opens TCP connection to remote BGP peer and authenticates sending BGP peer
  - **UPDATE**: advertises new path (or withdraws old)
  - **KEEPALIVE**: keeps connection alive in absence of UPDATES; also ACKs OPEN request
  - **NOTIFICATION**: reports errors in previous msg; also used to close connection



# Path attributes and BGP routes

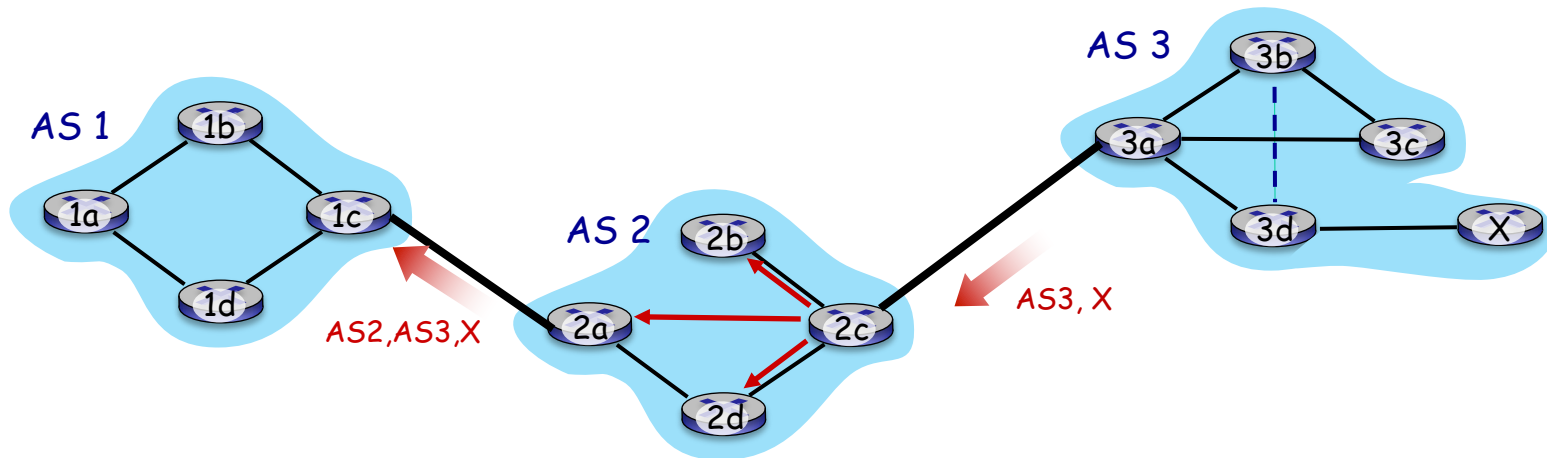
- BGP advertised route: prefix + attributes
  - prefix: destination being advertised
  - two important attributes:
    - **AS-PATH**: list of ASes through which prefix advertisement has passed
    - **NEXT-HOP**: indicates specific internal-AS router to next-hop AS
- **policy-based routing**:
  - gateway receiving route advertisement uses **import policy** to accept/decline path (e.g., never route through AS Y).
  - AS policy also determines whether to **advertise** path to other neighboring ASes







# BGP path advertisement

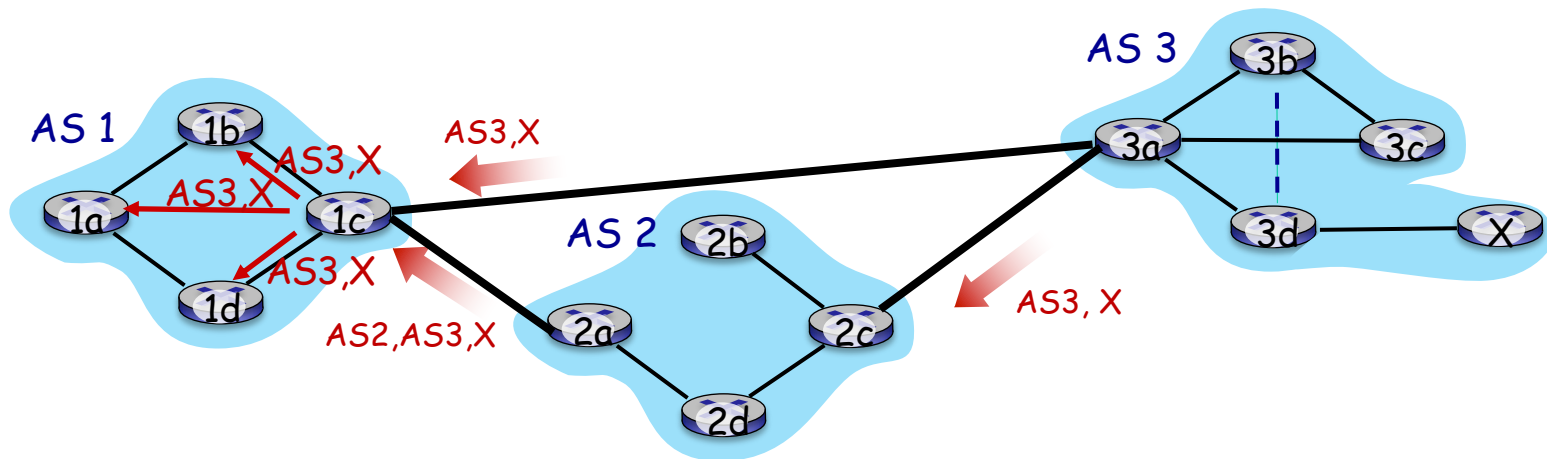


- AS2 router 2c receives path advertisement **AS3, X** (via eBGP) from AS3 router 3a
- based on AS2 policy, AS2 router 2c accepts path AS3, X, propagates (via iBGP) to all AS2 routers
- based on AS2 policy, AS2 router 2a advertises (via eBGP) path **AS2, AS3, X** to AS1 router 1c





# BGP path advertisement: multiple paths



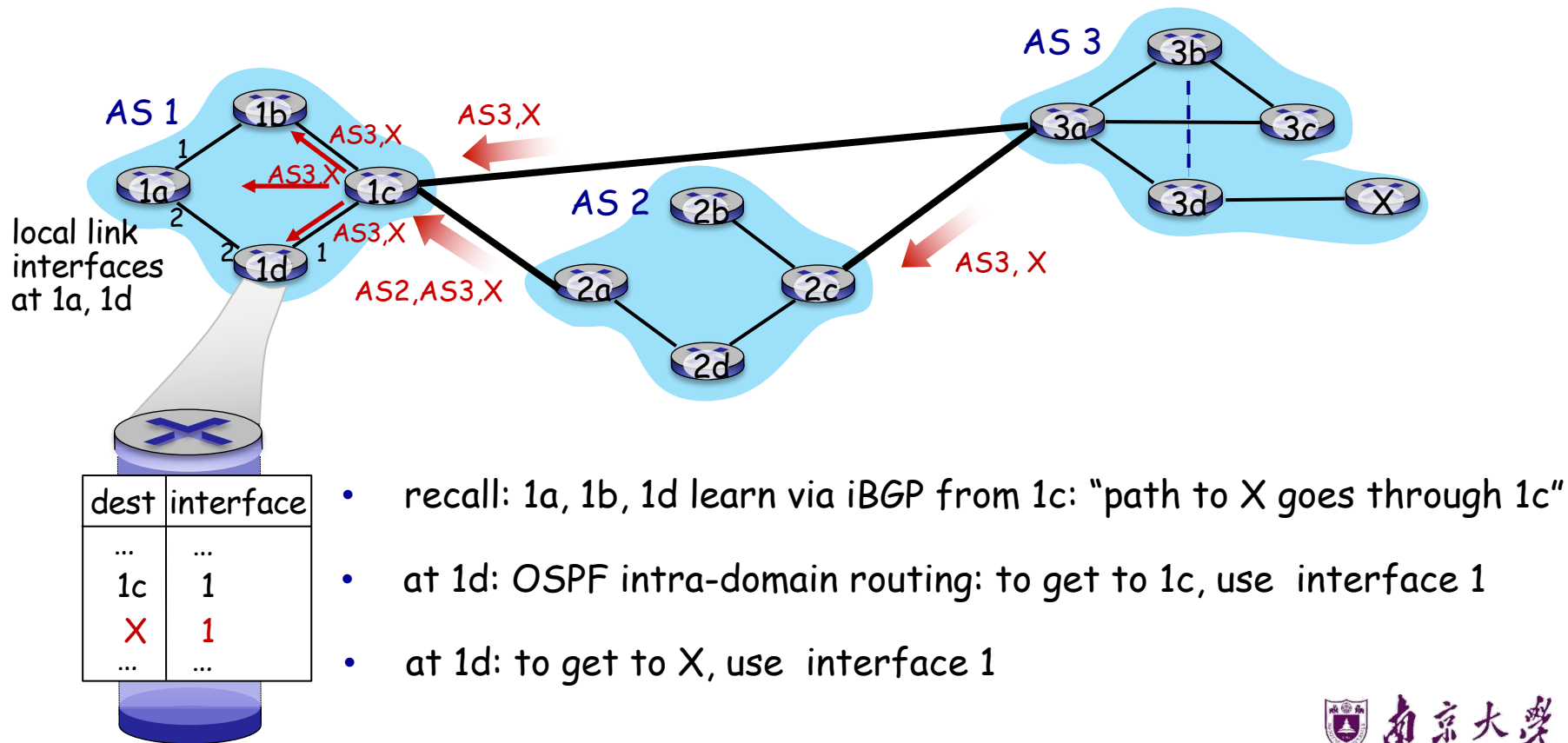
gateway router may learn about **multiple** paths to destination:

- AS1 gateway router 1c learns path **AS2,AS3,X** from 2a
- AS1 gateway router 1c learns path **AS3,X** from 3a
- based on **policy**, AS1 gateway router 1c chooses path **AS3,X** and advertises path within AS1 via iBGP



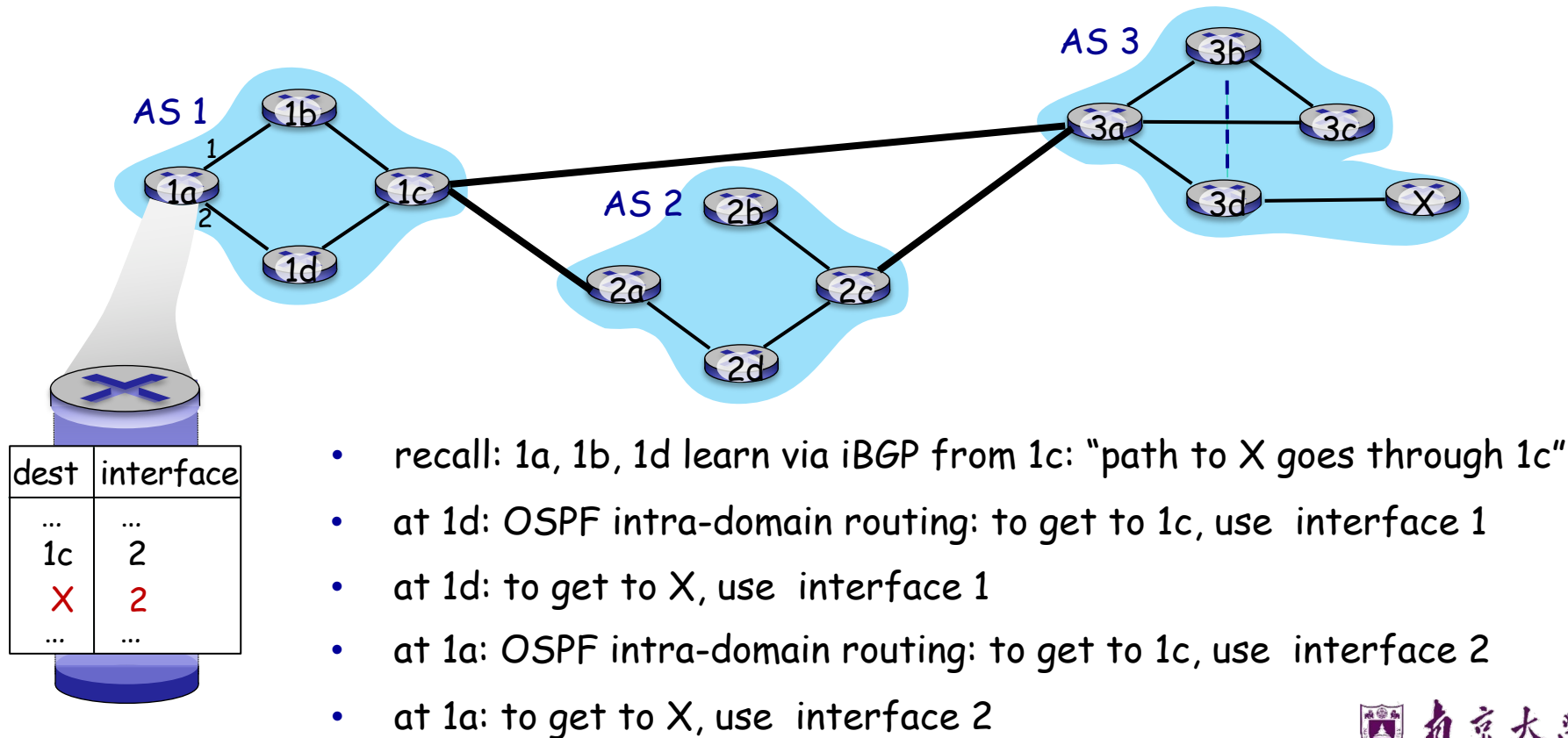


# BGP: populating forwarding tables



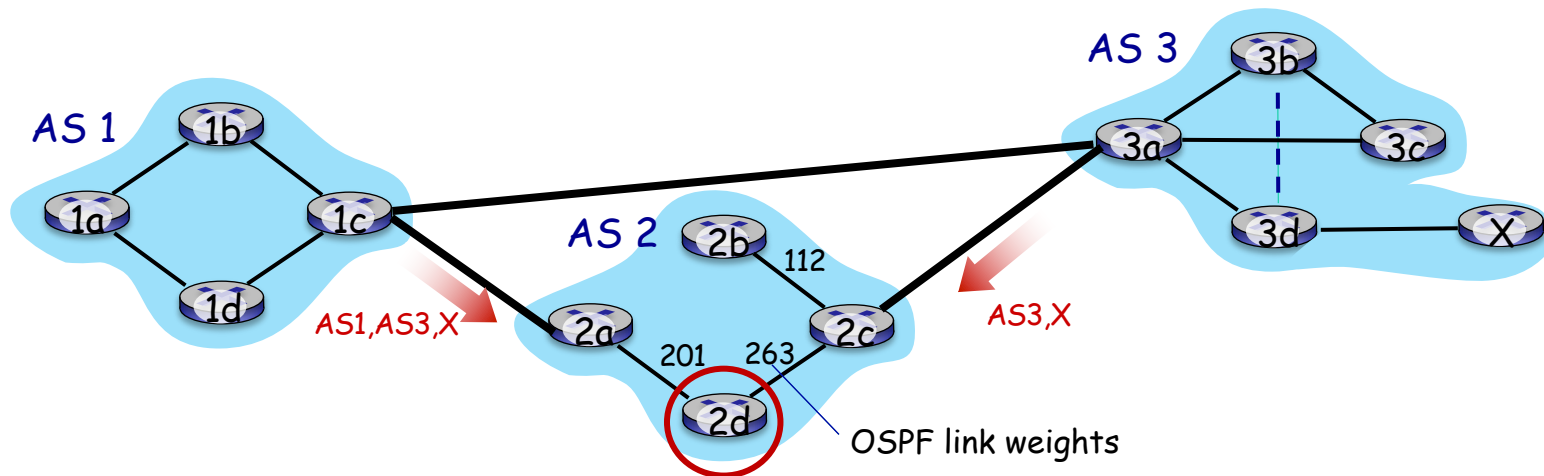


# BGP: populating forwarding tables





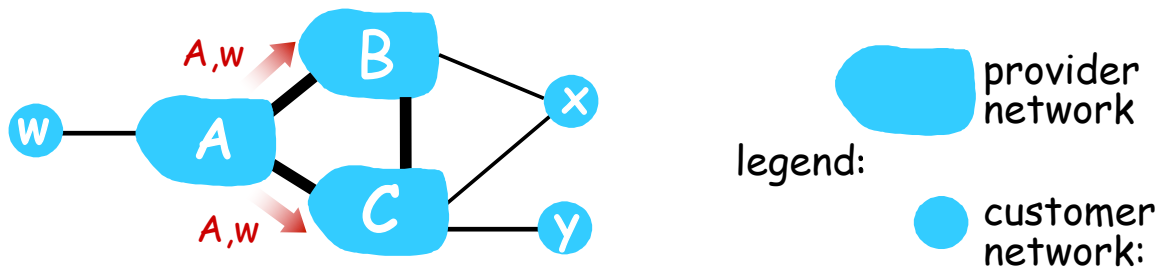
# Hot potato routing



- 2d learns (via iBGP) it can route to X via 2a or 2c
- **hot potato routing**: choose local gateway that has least intra-domain cost (e.g., 2d chooses 2a, even though more AS hops to X): don't worry about inter-domain cost!



# BGP: achieving policy via advertisements



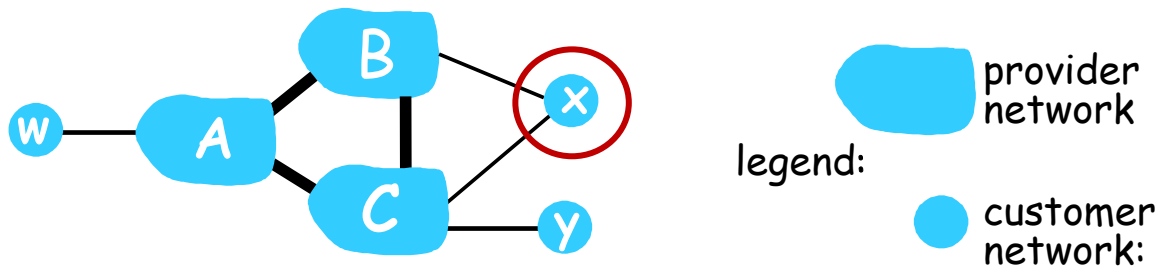
ISP only wants to route traffic to/from its customer networks (does not want to carry transit traffic between other ISPs - a typical "real world" policy)

- A advertises path Aw to B and to C
- B **chooses not to advertise** BA<sub>w</sub> to C!
  - B gets no "revenue" for routing CBA<sub>w</sub>, since none of C, A, w are B's customers
  - C does not learn about CBA<sub>w</sub> path
- C will route CA<sub>w</sub> (not using B) to get to w





# BGP: achieving policy via advertisements (more)



ISP only wants to route traffic to/from its customer networks (does not want to carry transit traffic between other ISPs - a typical “real world” policy)

- A,B,C are **provider networks**
- x,w,y are **customer** (of provider networks)
- x is **dual-homed**: attached to two networks
- **policy to enforce**: x does not want to route from B to C via x
  - .. so x will not advertise to B a route to C





# BGP route selection

---

- router may learn about more than one route to destination AS, selects route based on:
  1. local preference value attribute: policy decision
  2. shortest AS-PATH
  3. closest NEXT-HOP router: hot potato routing
  4. additional criteria







# Why different Intra-, Inter-AS routing ?

## policy:

- inter-AS: admin wants control over how its traffic routed, who routes through its network
- intra-AS: single admin, so policy less of an issue

## scale:

- hierarchical routing saves table size, reduced update traffic

## performance:

- intra-AS: can focus on performance
- inter-AS: policy dominates over performance





# Outline

---

- Routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol
- Network management, configuration





# Software defined networking (SDN)

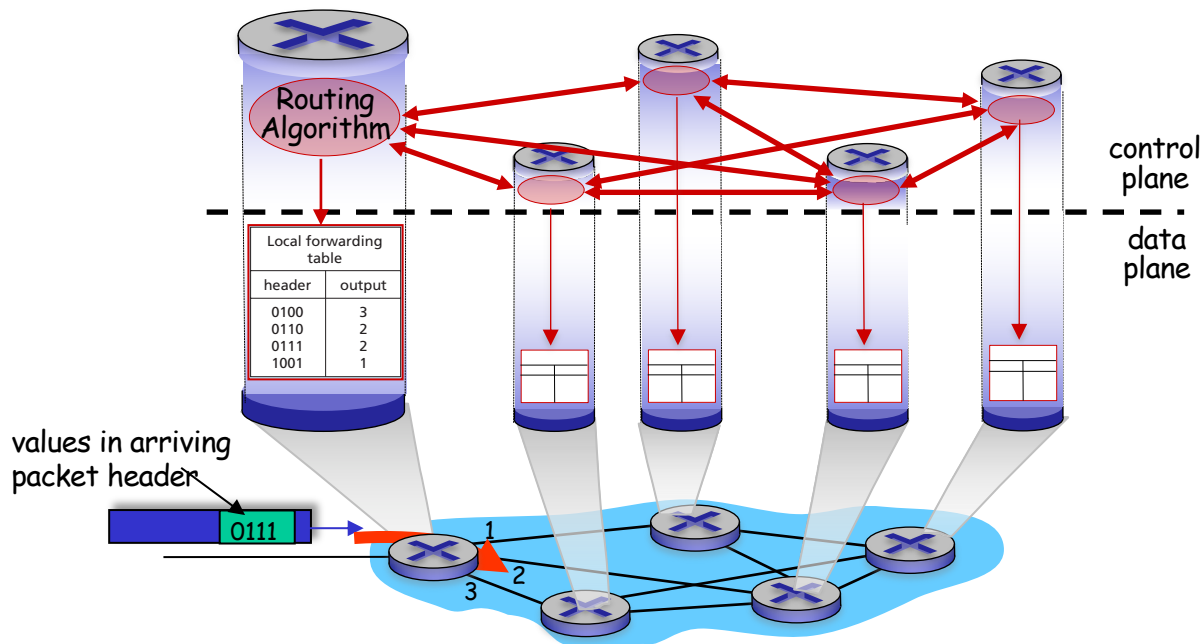
- Internet network layer: historically implemented via distributed, per-router control approach:
  - **monolithic** router contains switching hardware, runs proprietary implementation of Internet standard protocols (IP, RIP, IS-IS, OSPF, BGP) in proprietary router OS (e.g., Cisco IOS)
  - different “middleboxes” for different network layer functions: firewalls, load balancers, NAT boxes, ..
- ~2005: renewed interest in rethinking network control plane





# Per-router control plane

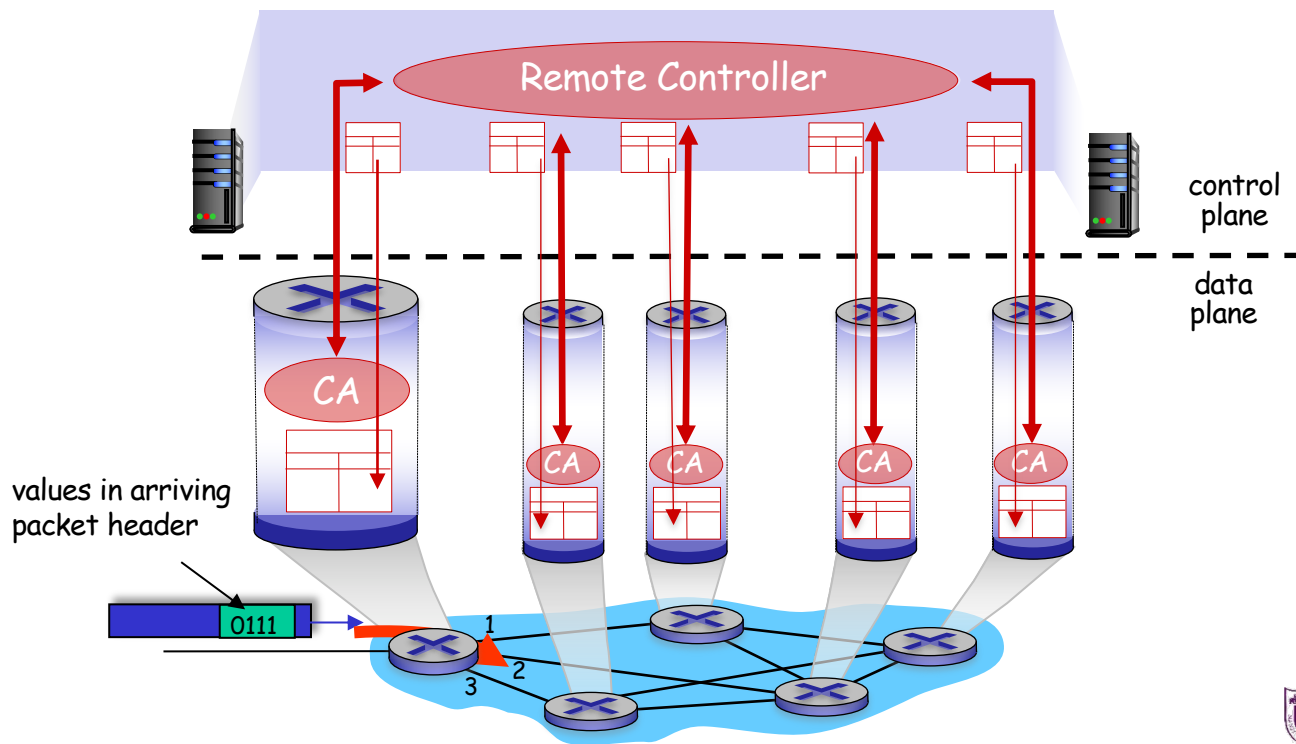
Individual routing algorithm components in each and every router interact in the control plane to compute forwarding tables





# Software-Defined Networking (SDN) control plane

Remote controller computes, installs forwarding tables in routers





# Software defined networking (SDN)

---

## Why a logically centralized control plane?

- easier network management: avoid router misconfigurations, greater flexibility of traffic flows
- table-based forwarding (recall OpenFlow API) allows “programming” routers
  - centralized “programming” easier: compute tables centrally and distribute
  - distributed “programming” more difficult: compute tables as result of distributed algorithm (protocol) implemented in each-and-every router
- open (non-proprietary) implementation of control plane
  - foster innovation: let 1000 flowers bloom

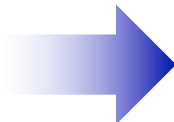
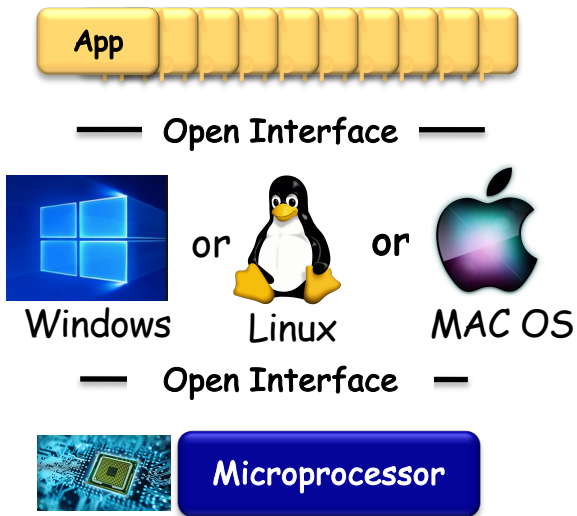
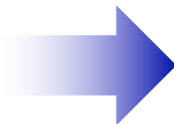




# SDN analogy: mainframe to PC revolution



Vertically integrated  
Closed, proprietary  
Slow innovation  
Small industry

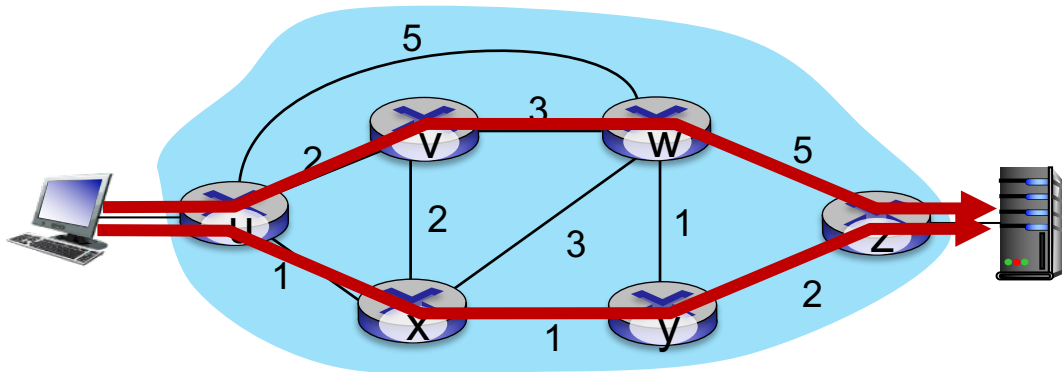


Horizontal  
Open interfaces  
Rapid innovation  
Huge industry





# Traffic engineering: difficult with traditional routing



**Q:** what if network operator wants u-to-z traffic to flow along **uvwz**, rather than **xyz**?

**A:** need to re-define link weights so traffic routing algorithm computes routes accordingly (or need a new routing algorithm)!

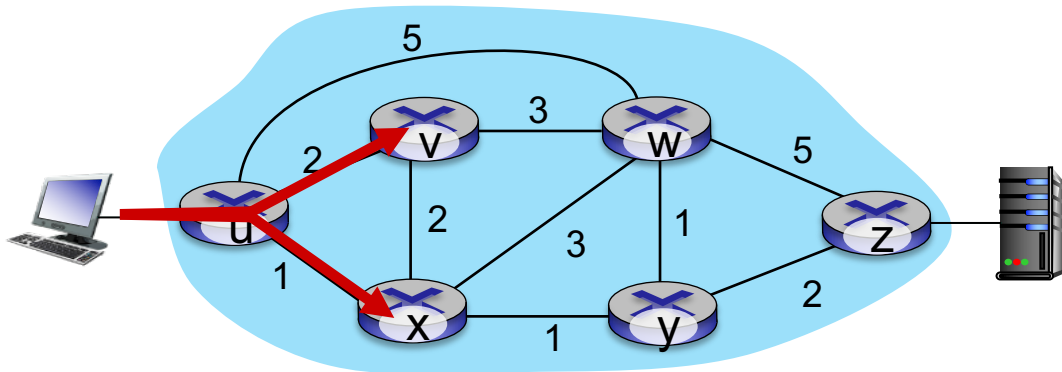
link weights are only control "knobs": not much control!







# Traffic engineering: difficult with traditional routing



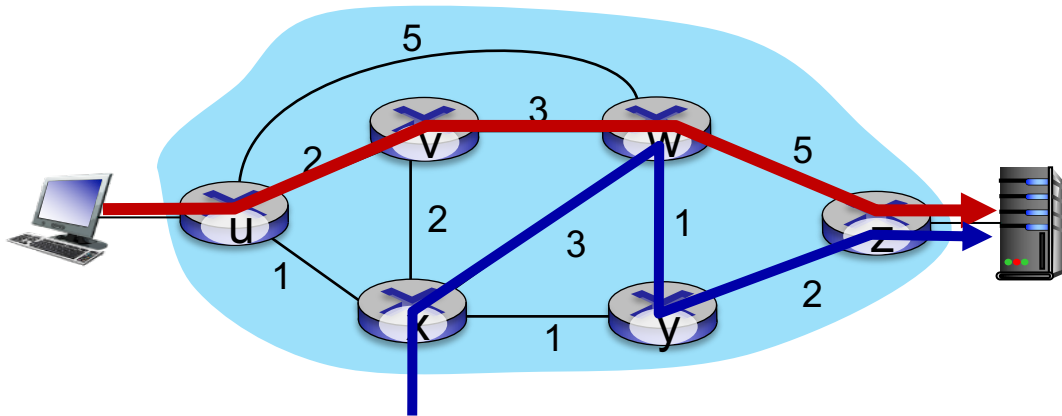
Q: what if network operator wants to split u-to-z traffic along **uvwz** and **uxyz** (load balancing)?

A: can't do it (or need a new routing algorithm)





# Traffic engineering: difficult with traditional routing



Q: what if w wants to route blue and red traffic differently from w to z?

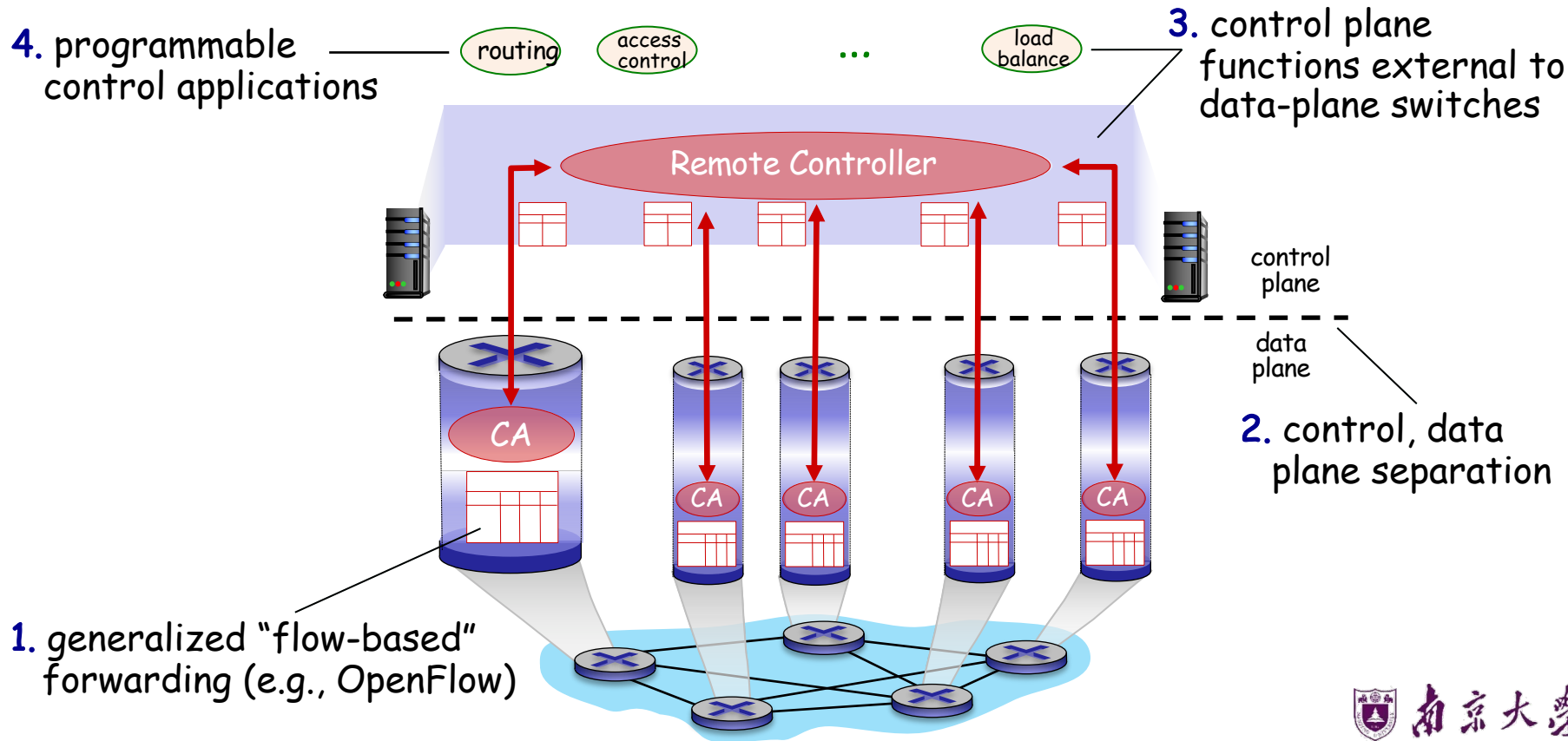
A: can't do it (with destination-based forwarding, and LS, DV routing)

We learned in Chapter 4 that generalized forwarding and SDN  
can be used to achieve any routing desired





# Software defined networking (SDN)

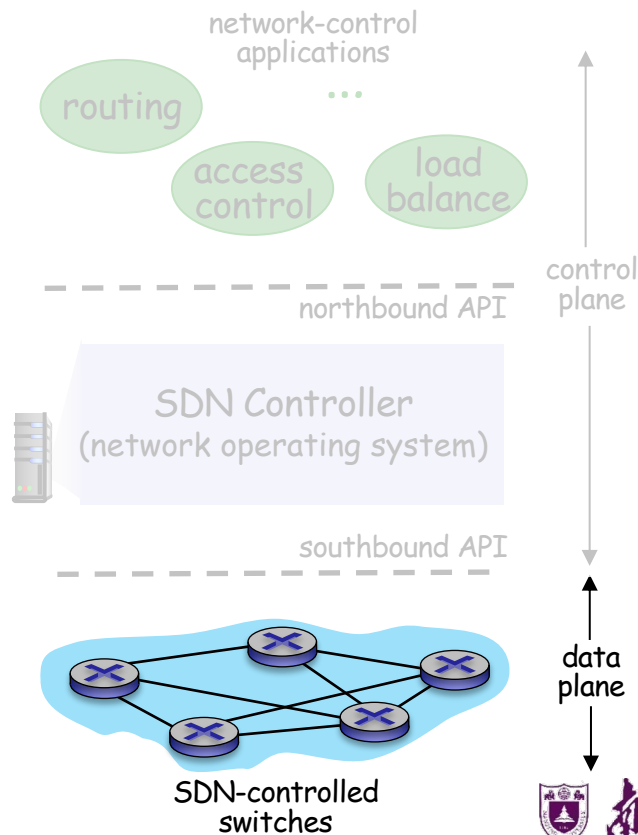




# Software defined networking (SDN)

## Data-plane switches:

- fast, simple, commodity switches implementing generalized data-plane forwarding (Section 4.4) in hardware
- flow (forwarding) table computed, installed under controller supervision
- API for table-based switch control (e.g., OpenFlow)
  - defines what is controllable, what is not
- protocol for communicating with controller (e.g., OpenFlow)

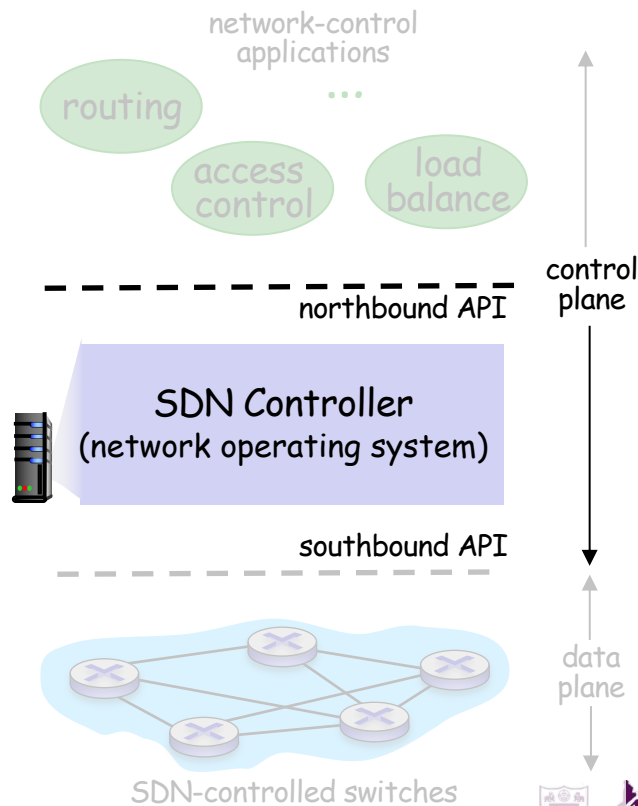




# Software defined networking (SDN)

## SDN controller (network OS):

- maintain network state information
- interacts with network control applications "above" via northbound API
- interacts with network switches "below" via southbound API
- implemented as distributed system for performance, scalability, fault-tolerance, robustness

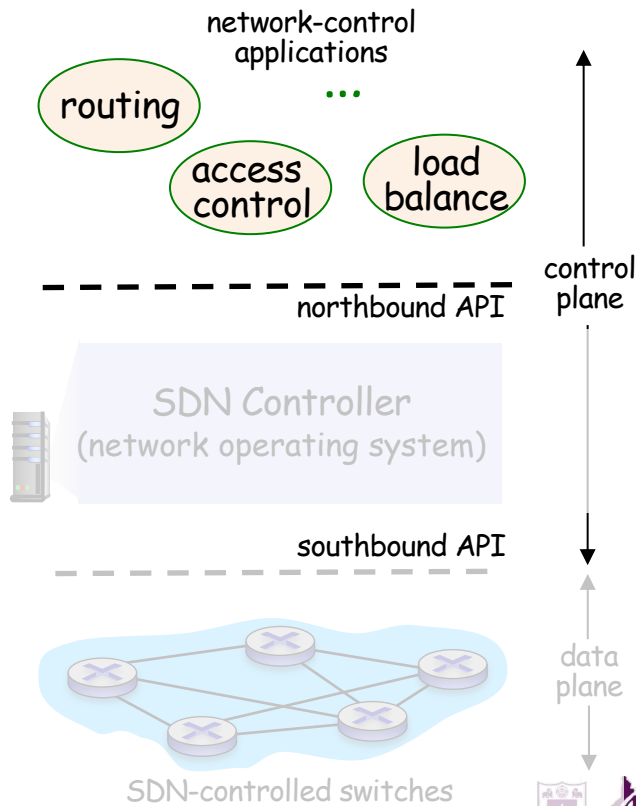




# Software defined networking (SDN)

## network-control apps:

- “brains” of control: implement control functions using lower-level services, API provided by SDN controller
- unbundled: can be provided by 3<sup>rd</sup> party: distinct from routing vendor, or SDN controller





# Components of SDN controller

interface layer to network control apps:

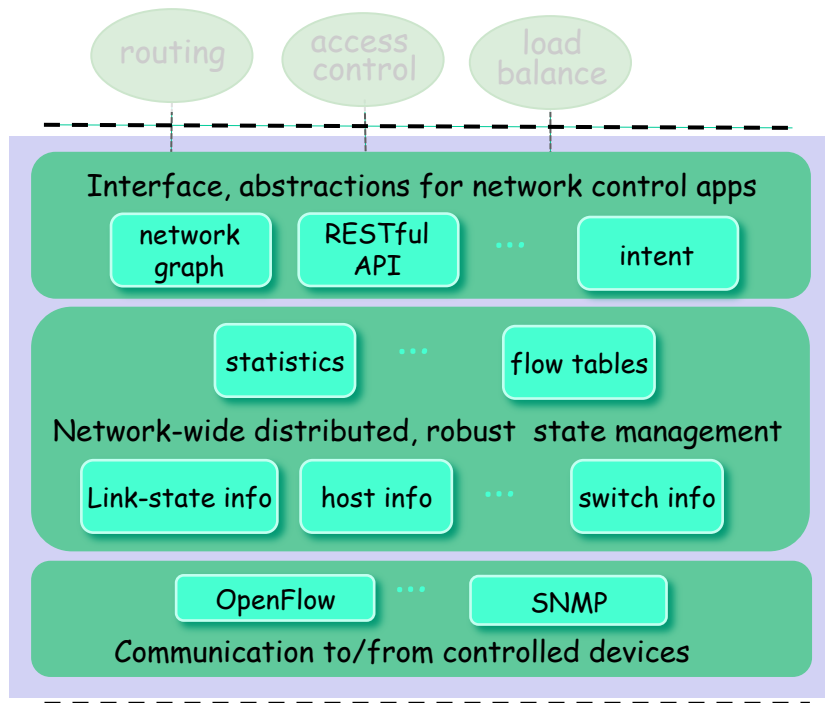
- abstractions API

network-wide state management :

- state of networks links, switches, services:  
a distributed database

communication:

- communicate between SDN controller and controlled switches



SDN controller

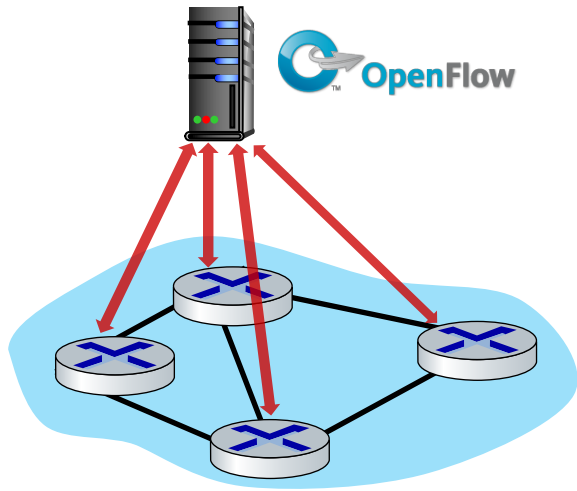




# OpenFlow protocol

- operates between controller, switch
- TCP used to exchange messages
  - optional encryption
- three classes of OpenFlow messages:
  - controller-to-switch
  - asynchronous (switch to controller)
  - symmetric (misc.)
- distinct from OpenFlow API
  - API used to specify generalized forwarding actions

## OpenFlow Controller





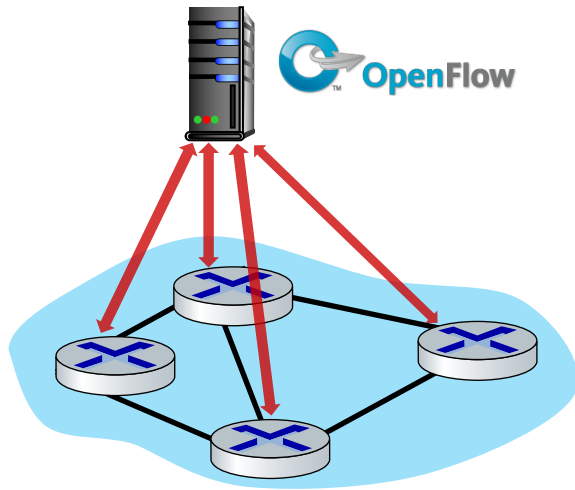


# OpenFlow: controller-to-switch messages

## Key controller-to-switch messages

- **features:** controller queries switch features, switch replies
- **configure:** controller queries/sets switch configuration parameters
- **modify-state:** add, delete, modify flow entries in the OpenFlow tables
- **packet-out:** controller can send this packet out of specific switch port

## OpenFlow Controller



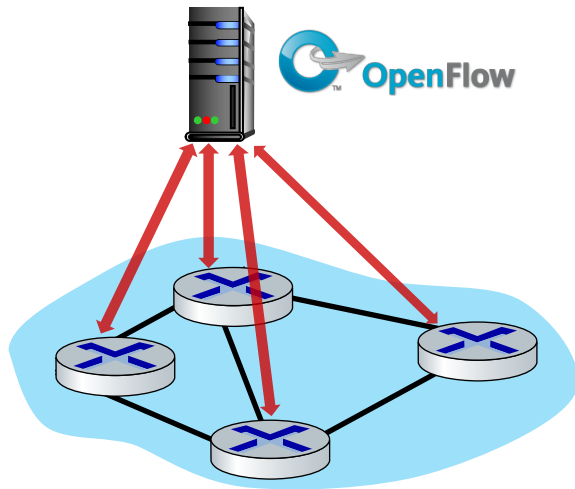


# OpenFlow: switch-to-controller messages

## Key switch-to-controller messages

- **packet-in**: transfer packet (and its control) to controller. See packet-out message from controller
- **flow-removed**: flow table entry deleted at switch
- **port status**: inform controller of a change on a port.

## OpenFlow Controller

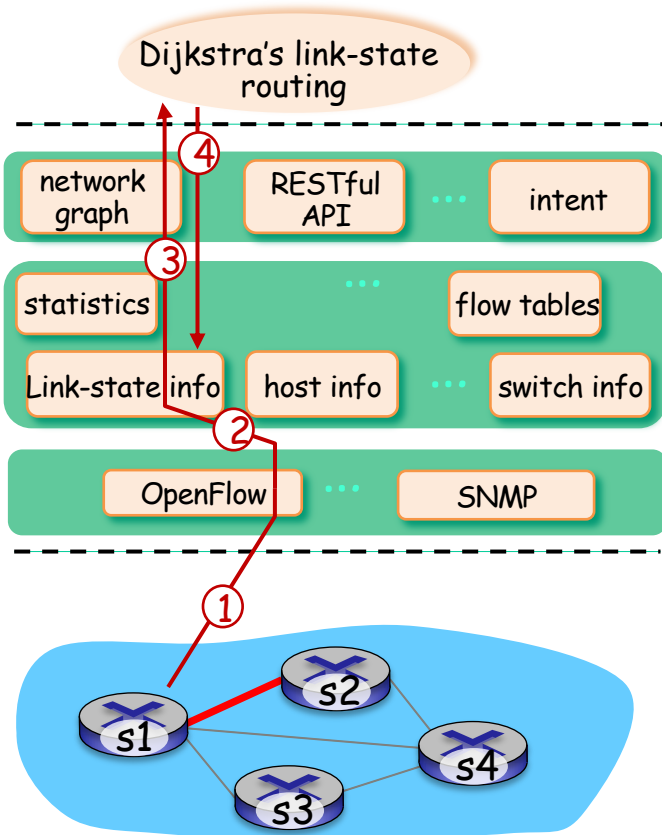


Fortunately, network operators don't "program" switches by creating/sending OpenFlow messages directly. Instead use higher-level abstraction at controller





# SDN: control/data plane interaction example

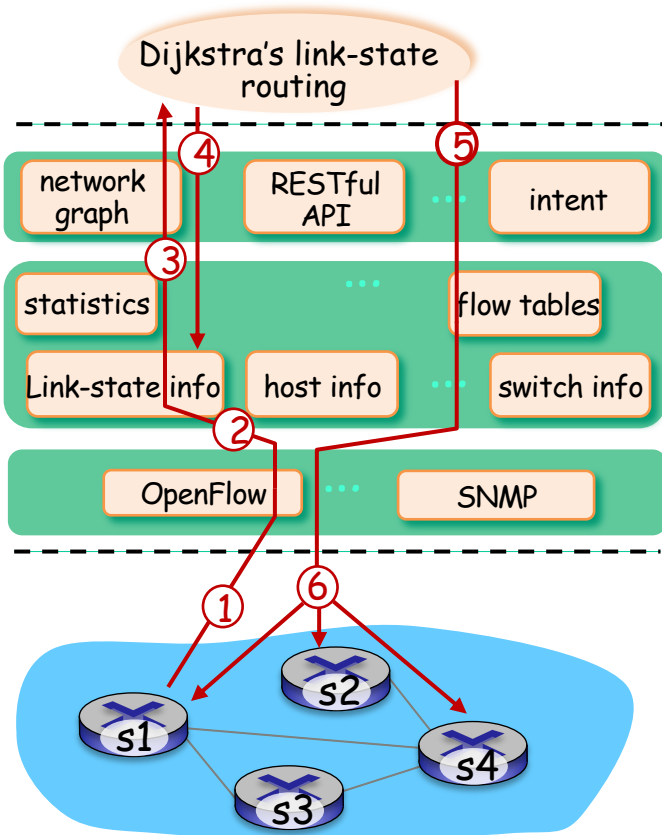


- ① S1, experiencing link failure uses OpenFlow port status message to notify controller
- ② SDN controller receives OpenFlow message, updates link status info
- ③ Dijkstra's routing algorithm application has previously registered to be called when ever link status changes. It is called.
- ④ Dijkstra's routing algorithm access network graph info, link state info in controller, computes new routes





# SDN: control/data plane interaction example



- ⑤ link state routing app interacts with flow-table-computation component in SDN controller, which computes new flow tables needed
- ⑥ controller uses OpenFlow to install new tables in switches that need updating



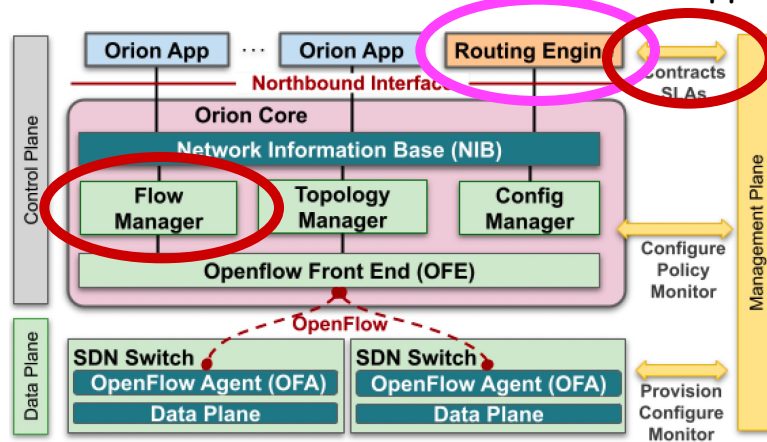


# Google ORION SDN control plane

**ORION:** Google's SDN control plane (*NSDI'21*): control plane for Google's datacenter (Jupiter) and wide area (B4) networks

- **routing** (intradomain, iBGP), traffic engineering: implemented in *applications* on top of ORION core
- **edge-edge flow-based** controls (e.g., CoFlow scheduling) to meet contract SLAs
- **management**: pub-sub distributed microservices in Orion core, OpenFlow for switch signaling/monitoring

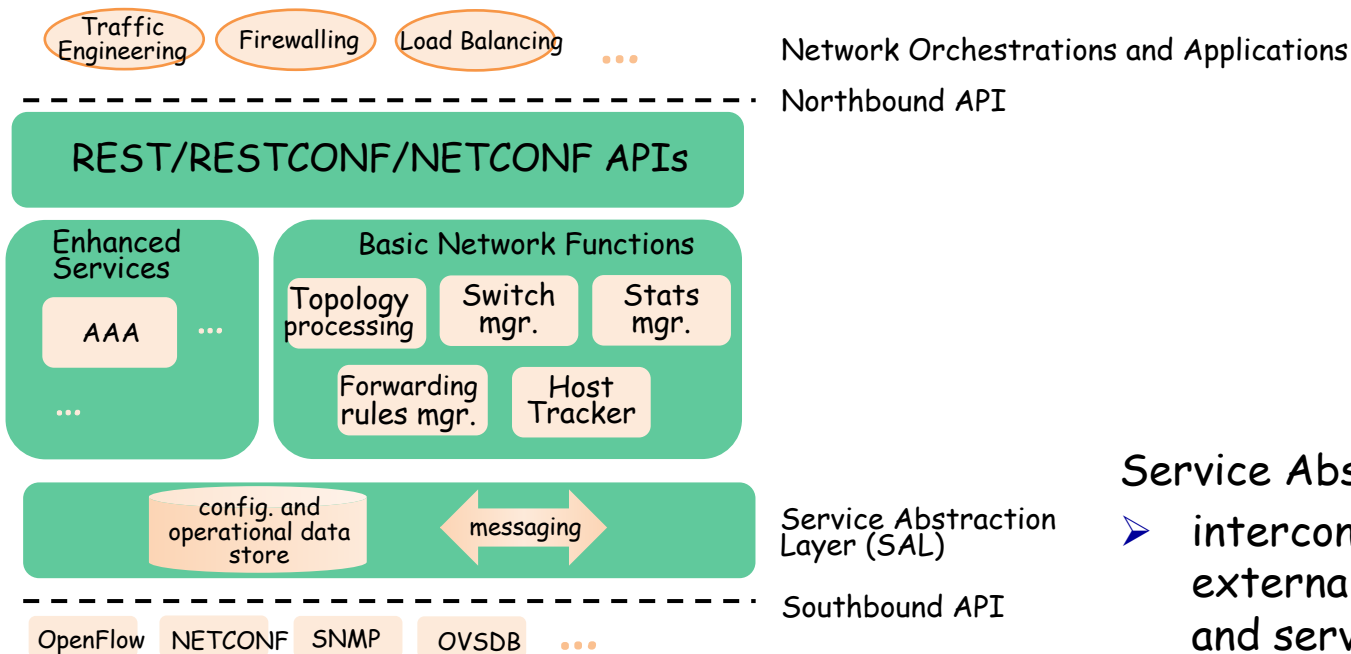
Orion SDN architecture and core apps



**Note:** ORION provides **intradomain** services within Google's network



# OpenDaylight (ODL) controller

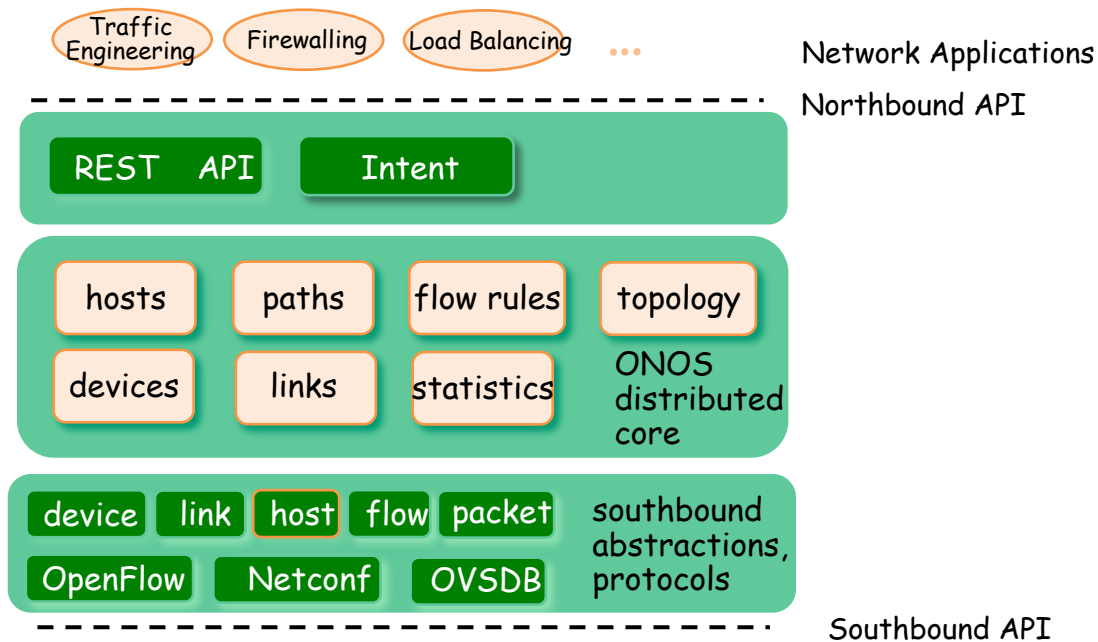


Service Abstraction Layer:  
➤ interconnects internal, external applications and services





# ONOS controller



- control apps separate from controller
- intent framework: high-level specification of service: what rather than how
- considerable emphasis on distributed core: service reliability, replication performance scaling



# SDN: selected challenges

---

- hardening the control plane: dependable, reliable, performance-scalable, secure distributed system
  - robustness to failures: leverage strong theory of reliable distributed system for control plane
  - dependability, security: “baked in” from day one?
- networks, protocols meeting mission-specific requirements
  - e.g., real-time, ultra-reliable, ultra-secure
- Internet-scaling: beyond a single AS
- SDN critical in 5G cellular networks







# SDN and the future of traditional network protocols

- SDN-computed versus router-computer forwarding tables:
  - just one example of logically-centralized-computed versus protocol computed
- one could imagine SDN-computed congestion control:
  - controller sets sender rates based on router-reported (to controller) congestion levels



How will implementation of network functionality (SDN versus protocols) evolve?





# Outline

---

- Routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol
- Network management, configuration





# ICMP: internet control message protocol

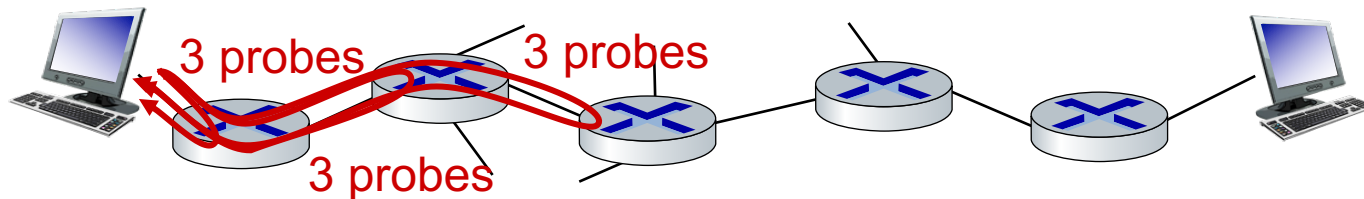
- used by hosts and routers to communicate network-level information
  - error reporting: unreachable host, network, port, protocol
  - echo request/reply (used by ping)
- network-layer "above" IP:
  - ICMP messages carried in IP datagrams
- **ICMP message:** type, code plus first 8 bytes of IP datagram causing error

| <u>Type</u> | <u>Code</u> | <u>description</u>                            |
|-------------|-------------|-----------------------------------------------|
| 0           | 0           | echo reply (ping)                             |
| 3           | 0           | dest. network unreachable                     |
| 3           | 1           | dest host unreachable                         |
| 3           | 2           | dest protocol unreachable                     |
| 3           | 3           | dest port unreachable                         |
| 3           | 6           | dest network unknown                          |
| 3           | 7           | dest host unknown                             |
| 4           | 0           | source quench (congestion control - not used) |
| 8           | 0           | echo request (ping)                           |
| 9           | 0           | route advertisement                           |
| 10          | 0           | router discovery                              |
| 11          | 0           | TTL expired                                   |
| 12          | 0           | bad IP header                                 |





# Traceroute and ICMP



- source sends sets of UDP segments to destination
    - 1<sup>st</sup> set has TTL =1, 2<sup>nd</sup> set has TTL=2, etc.
  - datagram in  $n$ th set arrives to  $n$ th router:
    - router discards datagram and sends source ICMP message (type 11, code 0)
    - ICMP message possibly includes name of router & IP address
  - when ICMP message arrives at source: record RTTs
- stopping criteria:
- UDP segment eventually arrives at destination host
  - destination returns ICMP “port unreachable” message (type 3, code 3)
  - source stops



# Outline

---

- Routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol
- Network management, configuration





# What is network management?

- autonomous systems (aka "network"): 1000s of interacting hardware/software components
- other complex systems requiring monitoring, configuration, control:
  - jet airplane, nuclear power plant, others?



"Network management includes the deployment, integration and coordination of the hardware, software, and human elements to monitor, test, poll, configure, analyze, evaluate, and control the network and element resources to meet the real-time, operational performance, and Quality of Service requirements at a reasonable cost."





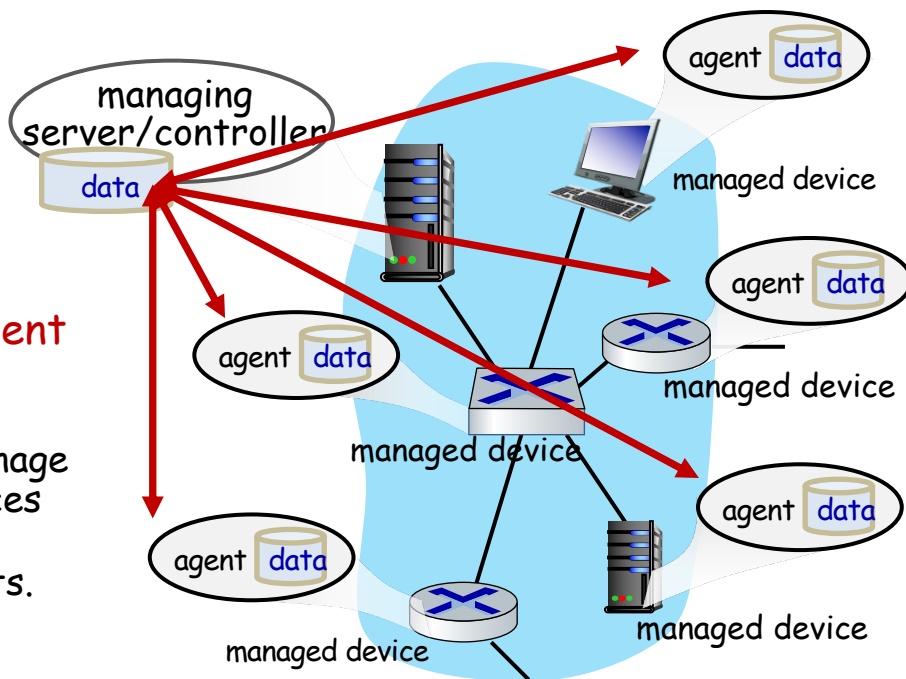
# Components of network management

## Managing server:

application, typically with network managers (humans) in the loop

## Network management protocol:

used by managing server to query, configure, manage device; used by devices to inform managing server of data, events.



## Managed device:

equipment with manageable, configurable hardware, software components

**Data:** device "state" configuration data, operational data, device statistics





# Network operator approaches to management

## CLI (Command Line Interface)

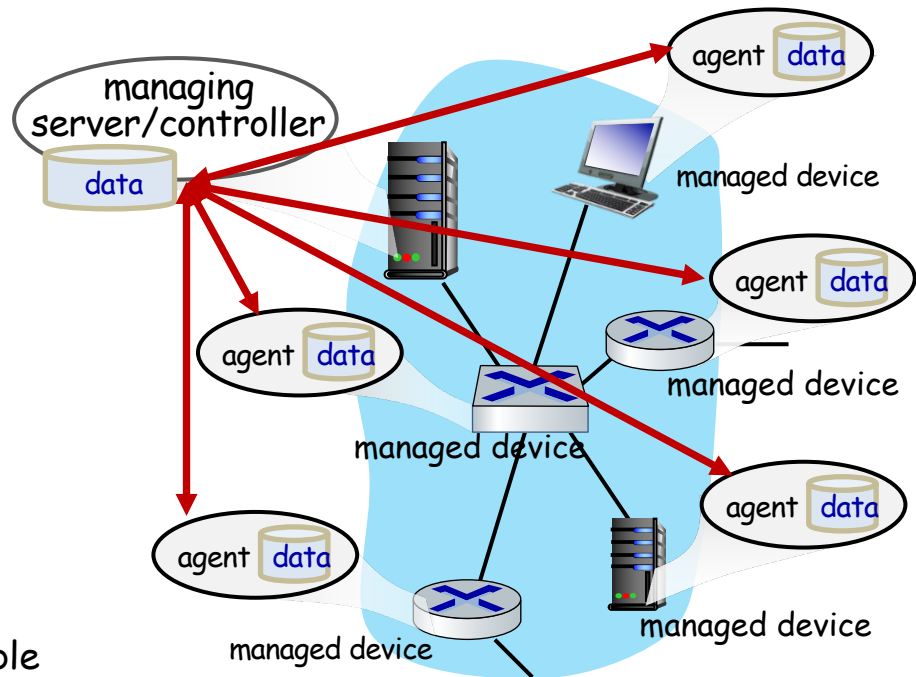
- operator issues (types, scripts) direct to individual devices (e.g., vis ssh)

## SNMP/MIB

- operator queries/sets devices data (MIB) using Simple Network Management Protocol (SNMP)

## NETCONF/YANG

- more abstract, network-wide, holistic
- emphasis on multi-device configuration management.
- YANG: data modeling language
- NETCONF: communicate YANG-compatible actions/data to/from/among remote devices

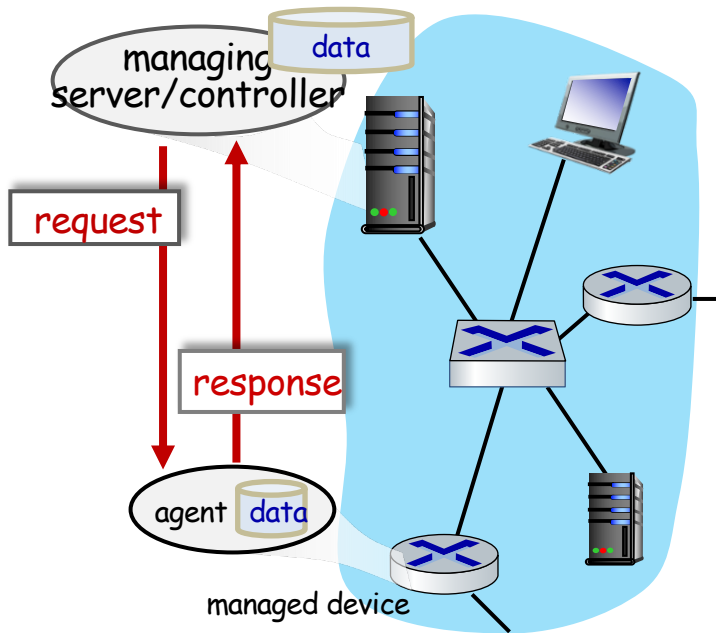




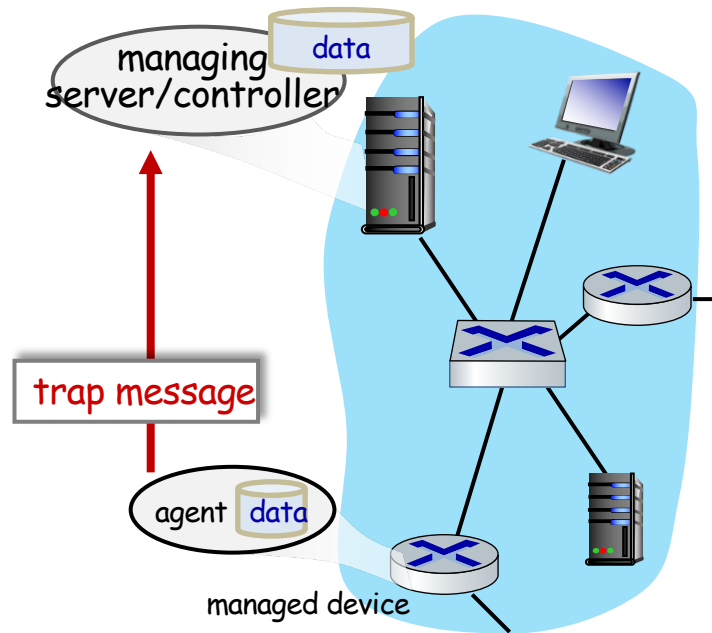


# SNMP protocol

Two ways to convey MIB info, commands:



request/response mode



trap mode





# SNMP protocol: message types

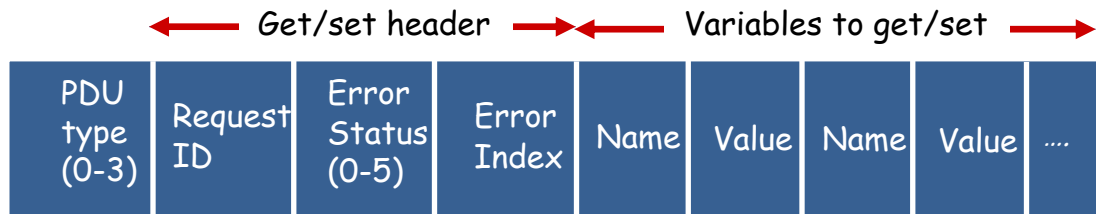
| Message type                                   | Function                                                                                 |
|------------------------------------------------|------------------------------------------------------------------------------------------|
| GetRequest<br>GetNextRequest<br>GetBulkRequest | manager-to-agent: “get me data”<br>(data instance, next data in list,<br>block of data). |
| SetRequest                                     | manager-to-agent: set MIB value                                                          |
| Response                                       | Agent-to-manager: value, response to Request                                             |
| Trap                                           | Agent-to-manager: inform manager of exceptional event                                    |



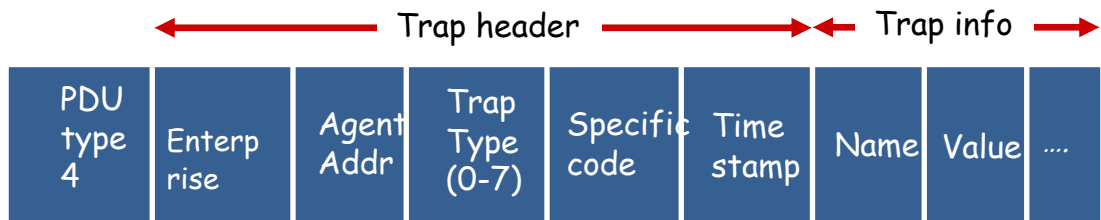


# SNMP protocol: message formats

message types 0-3



message type 4





# SNMP: Management Information Base (MIB)

- managed device's operational (and some configuration) data
- gathered into device **MIB module**
  - 400 MIB modules defined in RFC's; many more vendor-specific MIBs
- **Structure of Management Information (SMI):** data definition language
- example MIB variables for UDP protocol:



| Object ID       | Name            | Type           | Comments                                           |
|-----------------|-----------------|----------------|----------------------------------------------------|
| 1.3.6.1.2.1.7.1 | UDPInDatagrams  | 32-bit counter | total # datagrams delivered                        |
| 1.3.6.1.2.1.7.2 | UDPNoPorts      | 32-bit counter | # undeliverable datagrams (no application at port) |
| 1.3.6.1.2.1.7.3 | UDInErrors      | 32-bit counter | # undeliverable datagrams (all other reasons)      |
| 1.3.6.1.2.1.7.4 | UDPOutDatagrams | 32-bit counter | total # datagrams sent                             |
| 1.3.6.1.2.1.7.5 | udpTable        | SEQUENCE       | one entry for each port currently in use           |



# NETCONF overview

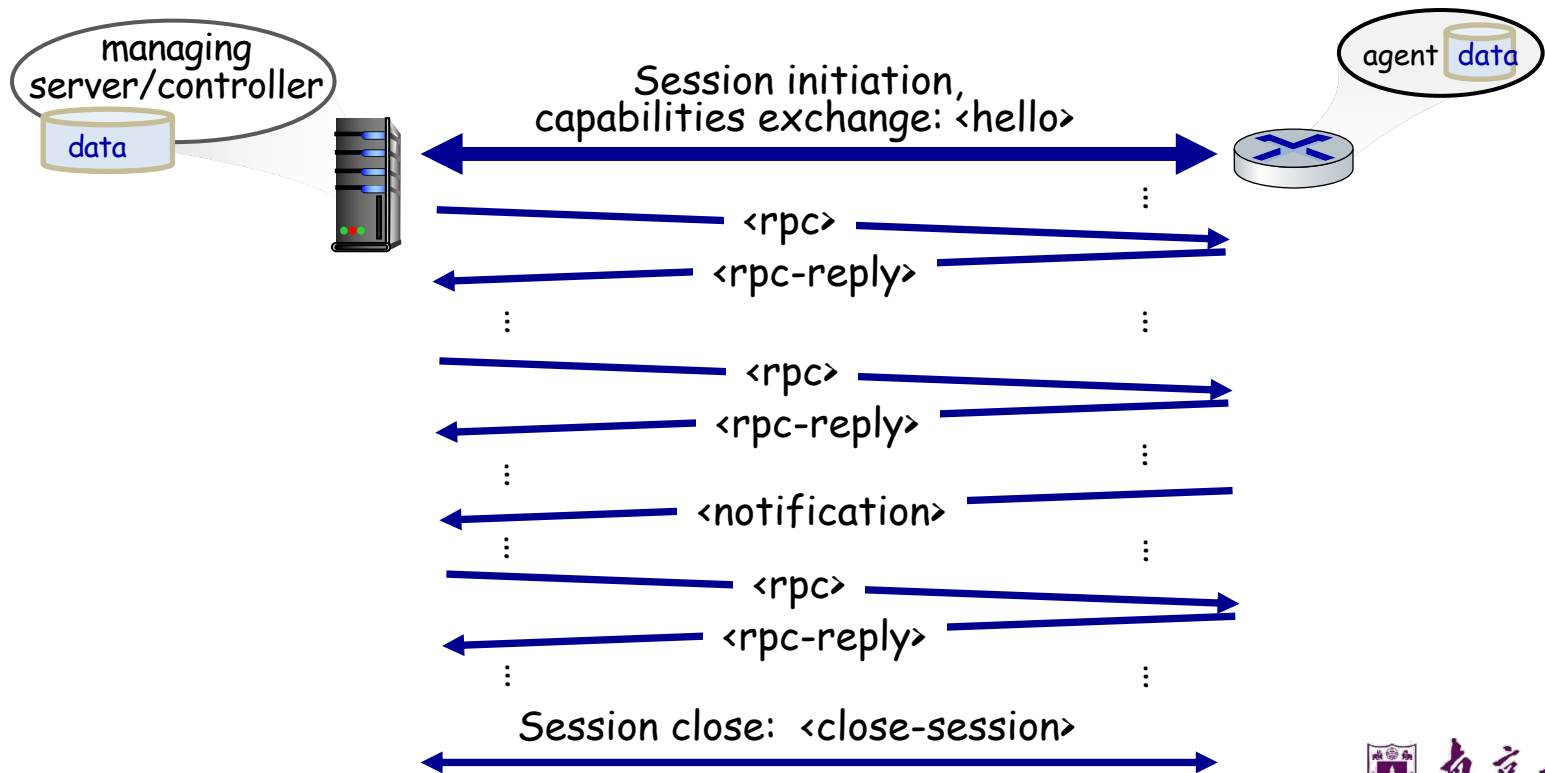
---

- **goal:** actively manage/**configure** devices network-wide
- operates between managing server and managed network devices
  - actions: retrieve, set, modify, activate configurations
  - **atomic-commit** actions over multiple devices
  - query operational data and statistics
  - subscribe to notifications from devices
- remote procedure call (RPC) paradigm
  - NETCONF protocol messages encoded in XML
  - exchanged over secure, reliable transport (e.g., TLS) protocol





# NETCONF initialization, exchange, close





# Selected NETCONF Operations

| NETCONF                               | Operation Description                                                                                                                             |
|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <get-config>                          | Retrieve all or part of <b>a given configuration</b> . A device may have multiple configurations.                                                 |
| <get>                                 | Retrieve all or part of both <b>configuration state and operational state</b> data.                                                               |
| <edit-config>                         | Change specified (possibly running) <b>configuration</b> at managed device. Managed device <rpc-reply> contains <ok> or <rpcerror> with rollback. |
| <lock>, <unlock>                      | Lock (unlock) <b>configuration datastore</b> at managed device (to lock out NETCONF, SNMP, or CLIs commands from other sources).                  |
| <create-subscription>, <notification> | Enable event <b>notification subscription</b> from managed device                                                                                 |





# Sample NETCONF RPC message

```
01 <?xml version="1.0" encoding="UTF-8"?>
02 <rpc message-id="101" note message id
03   xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
04   <edit-config change a configuration
05     <target>
06       <running/> change the running configuration
07     </target>
08     <config>
09       <top xmlns="http://example.com/schema/
10         1.2/config">
11         <interface>
12           <name>Ethernet0/0</name> change MTU of Ethernet 0/0 interface to 1500
13           <mtu>1500</mtu>
14         </interface>
15       </top>
16     </config>
17 </edit-config>
18 </rpc>
```

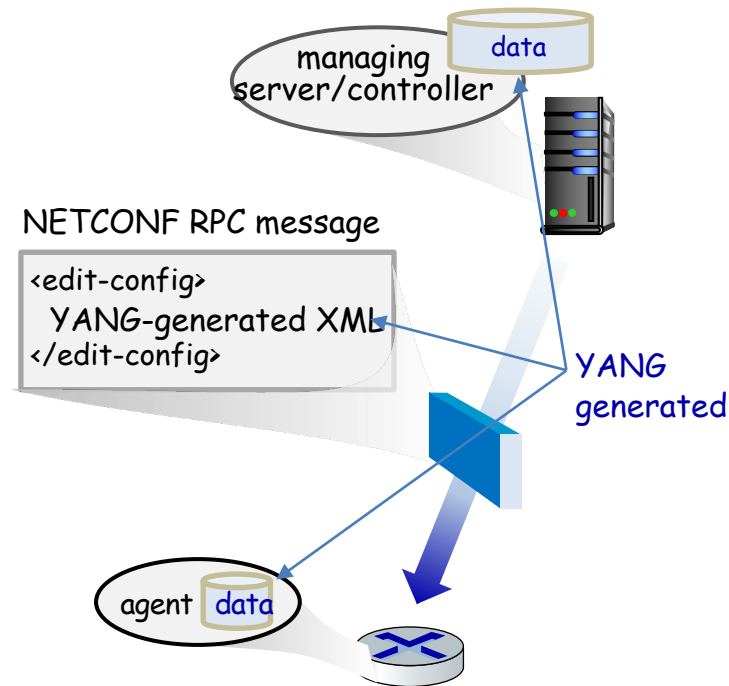






# YANG

- data modeling language used to specify structure, syntax, semantics of NETCONF network management data
  - built-in data types, like SMI
- XML document describing device, capabilities can be generated from YANG description
- can express constraints among data that must be satisfied by a valid NETCONF configuration
  - ensure NETCONF configurations satisfy correctness, consistency constraints





# Network layer: Summary

We've learned a lot!

- approaches to network control plane
  - per-router control (traditional)
  - logically centralized control (software defined networking)
- traditional routing algorithms
  - implementation in Internet: OSPF , BGP
- SDN controllers
  - implementation in practice: ODL, ONOS
- Internet Control Message Protocol
- network management

Next stop: link layer!





# 提问

## Q & A

孟晴开

智能软件与工程学院

苏州校区南雍楼西区228

qkmeng@nju.edu.cn , <https://mengqingkai.github.io/>



南京大學  
NANJING UNIVERSITY